

Task S4.01. Database Creation

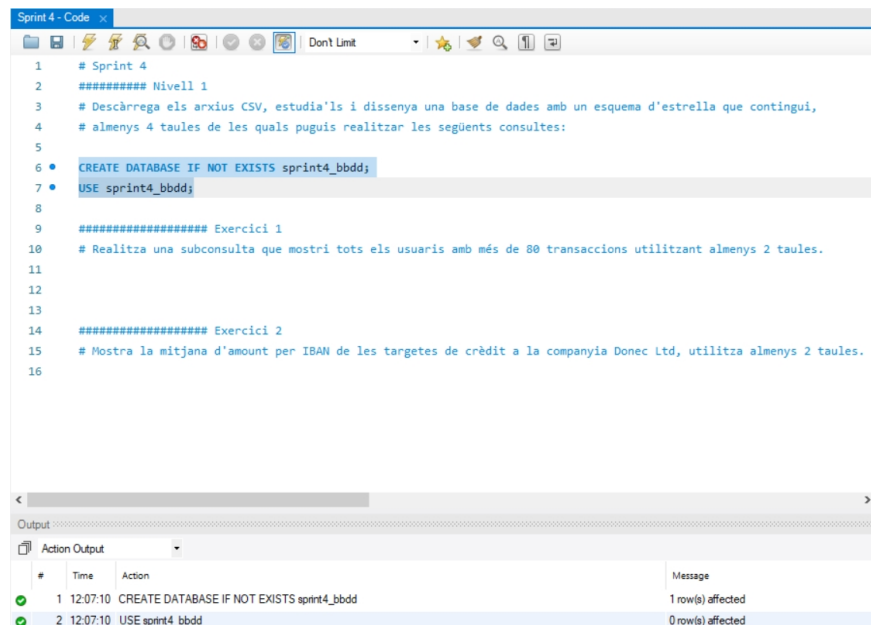
Mani Rezaeirad

p2p review partner: Damian Vallejos

Level 1

Download the CSV files, study them and design a database with a star schema that contains at least 4 tables from which you can perform the following queries:

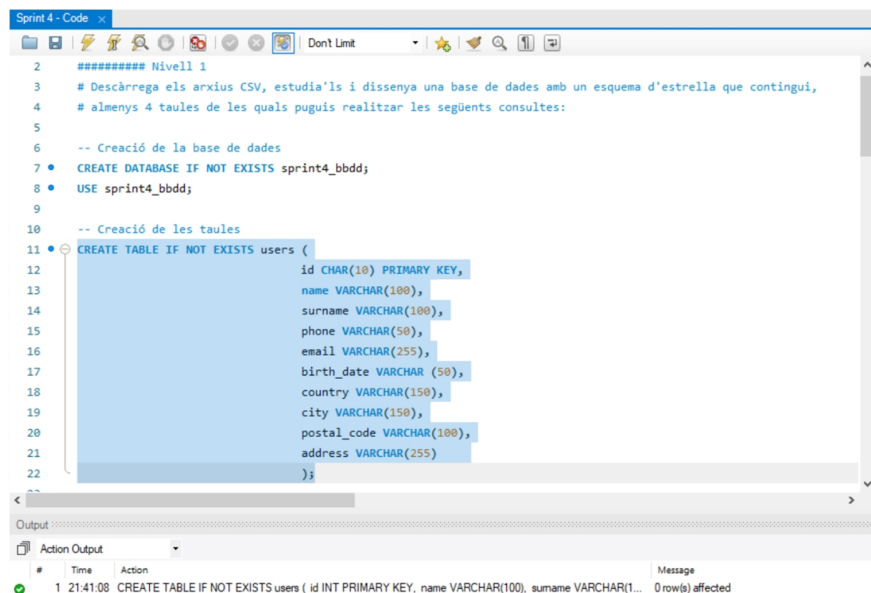
For starters, a database named **sprint4_bbdd** is created to host the tables that would be imported in the next step.



```
1 # Sprint 4
2 ##### Nivell 1
3 # Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui,
4 # almenys 4 taules de les quals puguis realitzar les següents consultes:
5
6 • CREATE DATABASE IF NOT EXISTS sprint4_bbdd;
7 • USE sprint4_bbdd;
8
9 ##### Exercici 1
10 # Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.
11
12
13
14 ##### Exercici 2
15 # Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.
16
```

#	Time	Action	Message
1	12:07:10	CREATE DATABASE IF NOT EXISTS sprint4_bbdd	1 row(s) affected
2	12:07:10	USE sprint4_bbdd	0 row(s) affected

Fig 1. Creating the database sprint4_bbdd



```
2 ##### Nivell 1
3 # Descàrrega els arxius CSV, estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui,
4 # almenys 4 taules de les quals puguis realitzar les següents consultes:
5
6 -- Creació de la base de dades
7 • CREATE DATABASE IF NOT EXISTS sprint4_bbdd;
8 • USE sprint4_bbdd;
9
10 -- Creació de les taules
11 • CREATE TABLE IF NOT EXISTS users (
12     id CHAR(10) PRIMARY KEY,
13     name VARCHAR(100),
14     surname VARCHAR(100),
15     phone VARCHAR(50),
16     email VARCHAR(255),
17     birth_date VARCHAR(50),
18     country VARCHAR(150),
19     city VARCHAR(150),
20     postal_code VARCHAR(100),
21     address VARCHAR(255)
22 );
```

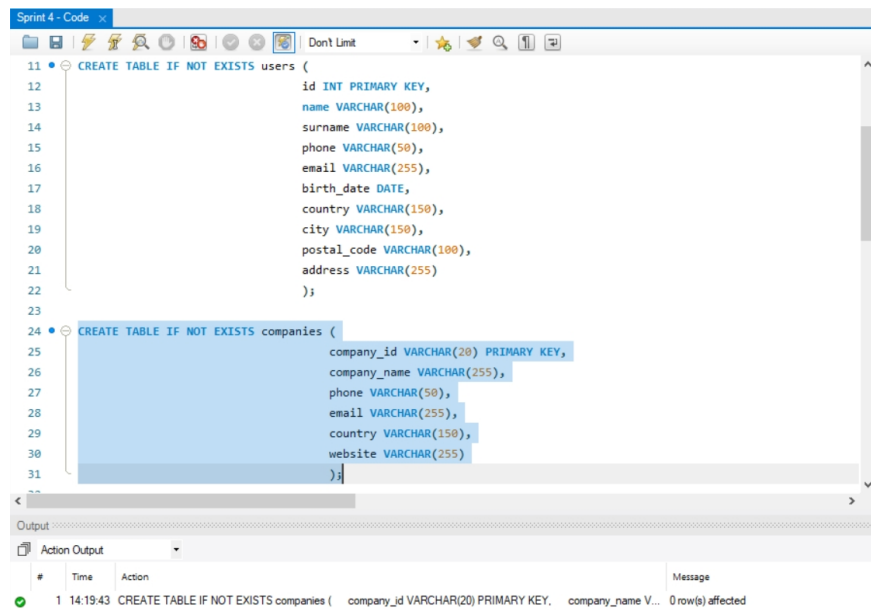
#	Time	Action	Message
1	21:41:08	CREATE TABLE IF NOT EXISTS users (id INT PRIMARY KEY, name VARCHAR(100), surname VARCHAR(100), phone VARCHAR(50), email VARCHAR(255), birth_date VARCHAR(50), country VARCHAR(150), city VARCHAR(150), postal_code VARCHAR(100), address VARCHAR(255));	0 row(s) affected

Fig 2. Creating the users table

Task S4.01. Database Creation

Mani Rezaeirad

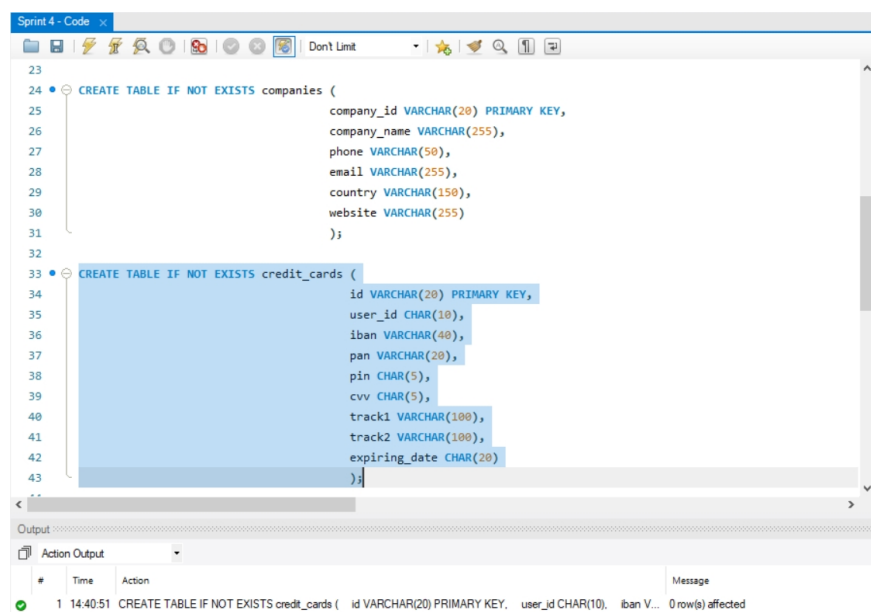
p2p review partner: Damian Vallejos



```
11 CREATE TABLE IF NOT EXISTS users (  
12     id INT PRIMARY KEY,  
13     name VARCHAR(100),  
14     surname VARCHAR(100),  
15     phone VARCHAR(50),  
16     email VARCHAR(255),  
17     birth_date DATE,  
18     country VARCHAR(150),  
19     city VARCHAR(150),  
20     postal_code VARCHAR(100),  
21     address VARCHAR(255)  
22 );  
23  
24 CREATE TABLE IF NOT EXISTS companies (  
25     company_id VARCHAR(20) PRIMARY KEY,  
26     company_name VARCHAR(255),  
27     phone VARCHAR(50),  
28     email VARCHAR(255),  
29     country VARCHAR(150),  
30     website VARCHAR(255)  
31 );
```

#	Time	Action	Message
1	14:19:43	CREATE TABLE IF NOT EXISTS companies (company_id VARCHAR(20) PRIMARY KEY, company_name V...	0 row(s) affected

Fig 3. Creating the companies table



```
23  
24 CREATE TABLE IF NOT EXISTS companies (  
25     company_id VARCHAR(20) PRIMARY KEY,  
26     company_name VARCHAR(255),  
27     phone VARCHAR(50),  
28     email VARCHAR(255),  
29     country VARCHAR(150),  
30     website VARCHAR(255)  
31 );  
32  
33 CREATE TABLE IF NOT EXISTS credit_cards (  
34     id VARCHAR(20) PRIMARY KEY,  
35     user_id CHAR(10),  
36     iban VARCHAR(40),  
37     pan VARCHAR(20),  
38     pin CHAR(5),  
39     cvv CHAR(5),  
40     track1 VARCHAR(100),  
41     track2 VARCHAR(100),  
42     expiring_date CHAR(20)  
43 );
```

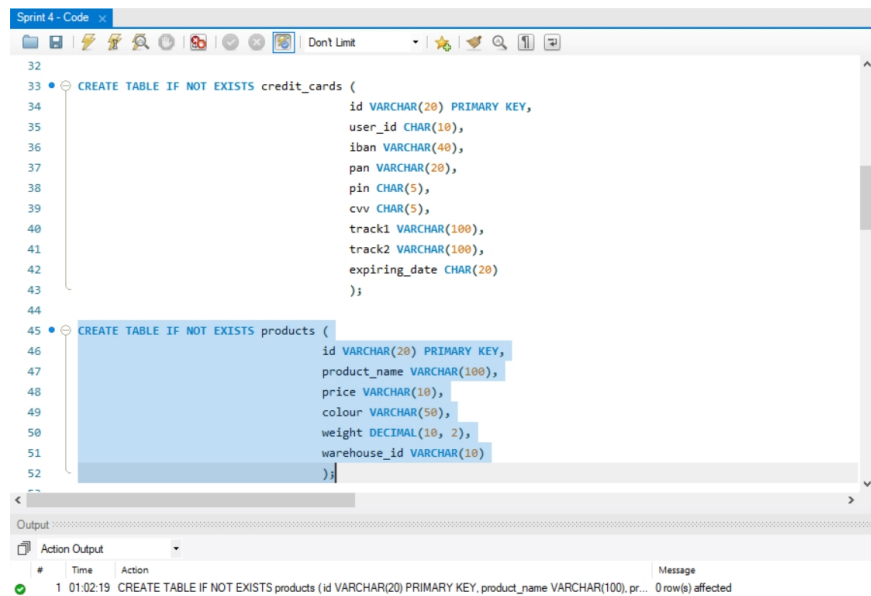
#	Time	Action	Message
1	14:40:51	CREATE TABLE IF NOT EXISTS credit_cards (id VARCHAR(20) PRIMARY KEY, user_id CHAR(10), iban V...	0 row(s) affected

Fig 4. Creating the credit_cards table

Task S4.01. Database Creation

Mani Rezaeirad

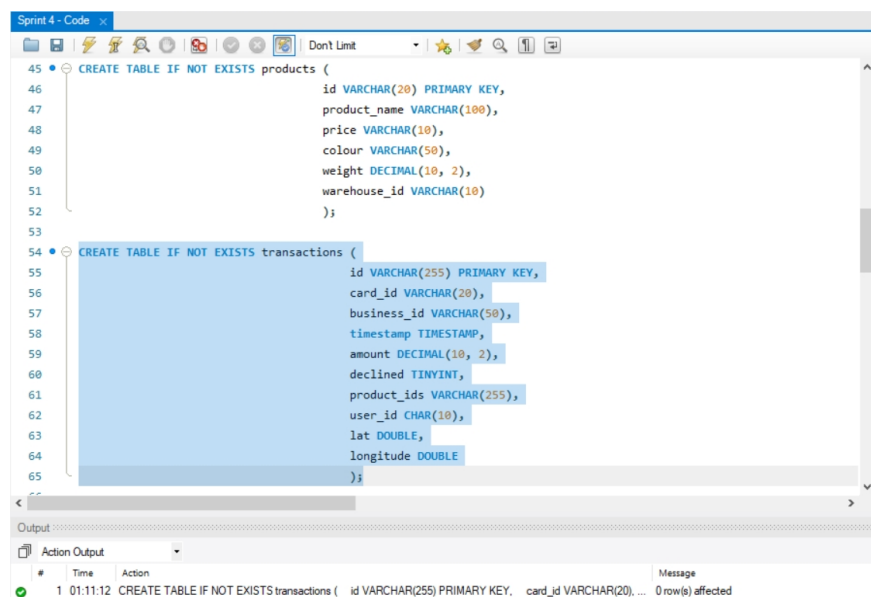
p2p review partner: Damian Vallejos



```
32
33 • CREATE TABLE IF NOT EXISTS credit_cards (
34     id VARCHAR(20) PRIMARY KEY,
35     user_id CHAR(10),
36     iban VARCHAR(40),
37     pan VARCHAR(20),
38     pin CHAR(5),
39     cvv CHAR(5),
40     track1 VARCHAR(100),
41     track2 VARCHAR(100),
42     expiring_date CHAR(20)
43 );
44
45 • CREATE TABLE IF NOT EXISTS products (
46     id VARCHAR(20) PRIMARY KEY,
47     product_name VARCHAR(100),
48     price VARCHAR(10),
49     colour VARCHAR(50),
50     weight DECIMAL(10, 2),
51     warehouse_id VARCHAR(10)
52 );
```

#	Time	Action	Message
1	01:02:19	CREATE TABLE IF NOT EXISTS products (id VARCHAR(20) PRIMARY KEY, product_name VARCHAR(100), pr...	0 row(s) affected

Fig 5. Creating the products table



```
45 • CREATE TABLE IF NOT EXISTS products (
46     id VARCHAR(20) PRIMARY KEY,
47     product_name VARCHAR(100),
48     price VARCHAR(10),
49     colour VARCHAR(50),
50     weight DECIMAL(10, 2),
51     warehouse_id VARCHAR(10)
52 );
53
54 • CREATE TABLE IF NOT EXISTS transactions (
55     id VARCHAR(255) PRIMARY KEY,
56     card_id VARCHAR(20),
57     business_id VARCHAR(50),
58     timestamp TIMESTAMP,
59     amount DECIMAL(10, 2),
60     declined TINYINT,
61     product_ids VARCHAR(255),
62     user_id CHAR(10),
63     lat DOUBLE,
64     longitude DOUBLE
65 );
```

#	Time	Action	Message
1	01:11:12	CREATE TABLE IF NOT EXISTS transactions (id VARCHAR(255) PRIMARY KEY, card_id VARCHAR(20), ...	0 row(s) affected

Fig 6. Creating the transactions table

Once the corresponding tables are created, it's time to import CSV files into them.

As shown in Figure 7, MySQL has a limitation that restricts the directories from which, files can be loaded. Figure 8 shows a command to display the configured directory. To solve this problem, there are two options: to move the CSV files to the specified directory or to disable the configuration. Since this change is just needed for this Sprint, it is better to simply move the CSV files to the designated directory. By doing so, the issue will be resolved, and loading the data can be done as shown in Figure 9 to Figure 14. Keep in mind that both CSV files, `american_users` and `europaean_users`, are imported into one single table: `users`.

Task S4.01. Database Creation

Mani Rezaeirad

p2p review partner: Damian Vallejos

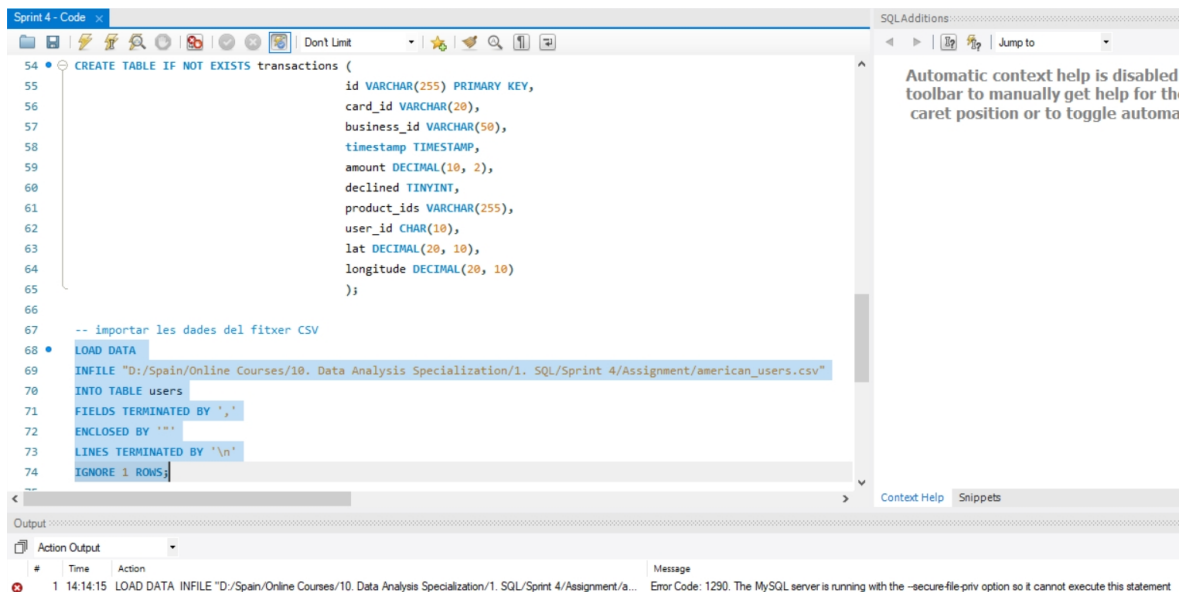


Fig 7. The directory limit prevents the loading of data files

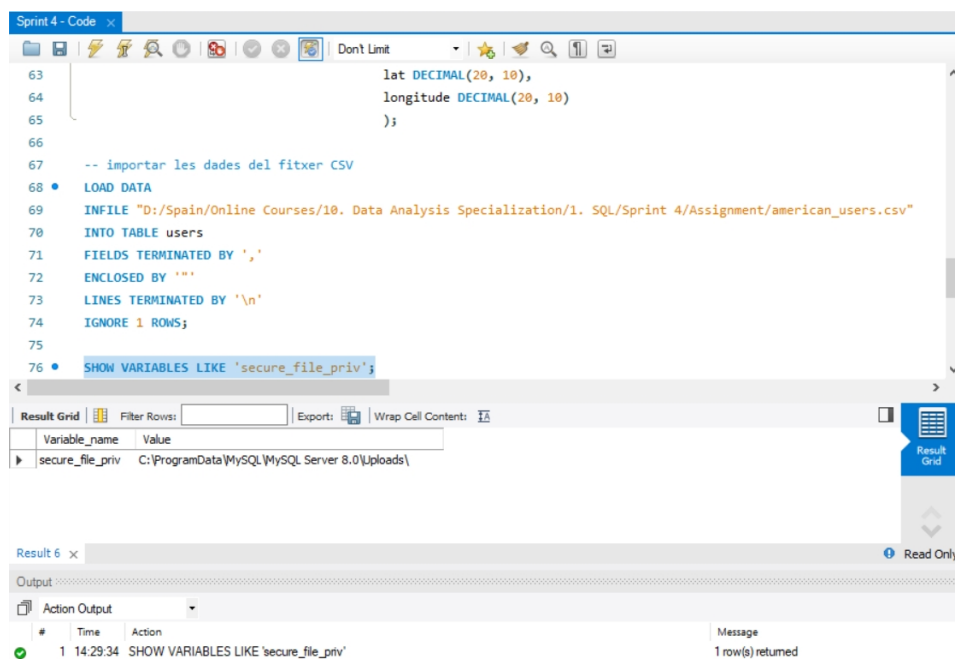
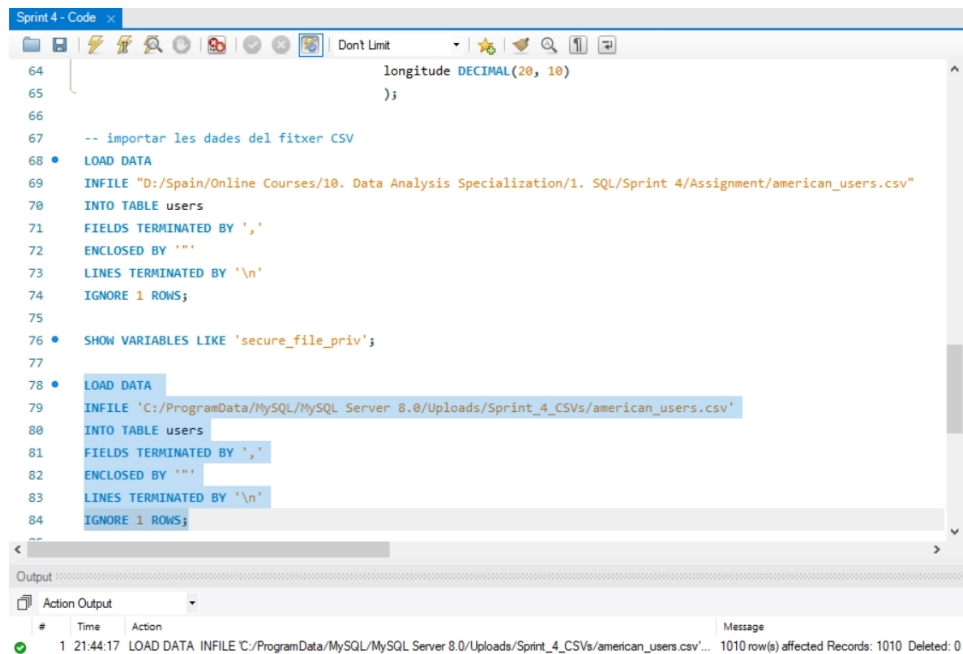


Fig 8. Showing the configured directory

Task S4.01. Database Creation

Mani Rezaeirad

p2p review partner: Damian Vallejos



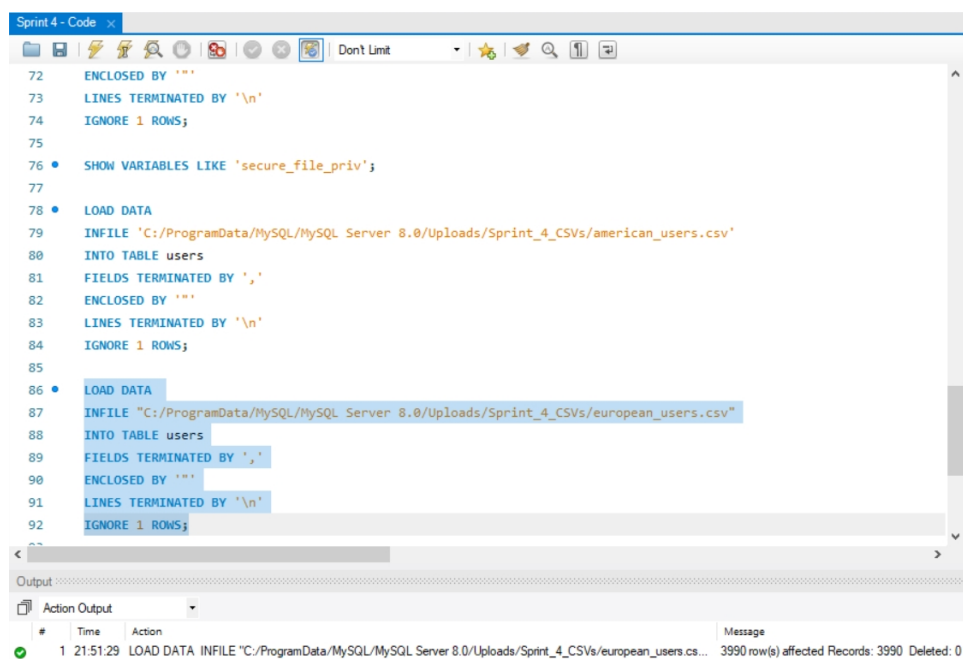
The screenshot shows a SQL IDE window titled "Sprint 4 - Code". The code editor contains SQL commands for importing a CSV file. The first part of the code (lines 64-77) defines a column and imports data from a local file. The second part (lines 78-84) imports data from a file located in the MySQL server's upload directory. The output window at the bottom shows a successful execution of the second command, indicating that 1010 rows were affected.

```
64 longitude DECIMAL(20, 10)
65 );
66
67 -- importar les dades del fitxer CSV
68 • LOAD DATA
69 INFILE "D:/Spain/Online Courses/10. Data Analysis Specialization/1. SQL/Sprint 4/Assignment/american_users.csv"
70 INTO TABLE users
71 FIELDS TERMINATED BY ','
72 ENCLOSED BY '"'
73 LINES TERMINATED BY '\n'
74 IGNORE 1 ROWS;
75
76 • SHOW VARIABLES LIKE 'secure_file_priv';
77
78 • LOAD DATA
79 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/american_users.csv'
80 INTO TABLE users
81 FIELDS TERMINATED BY ','
82 ENCLOSED BY '"'
83 LINES TERMINATED BY '\n'
84 IGNORE 1 ROWS;
```

Output

#	Time	Action	Message
1	21:44:17	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/american_users.csv'...	1010 row(s) affected Records: 1010 Deleted: 0

Fig 9. Importing the CSV data of american_users into the users table



The screenshot shows the same SQL IDE window with the code editor displaying SQL commands for importing a CSV file. The first part of the code (lines 72-77) is the same as in Figure 9. The second part (lines 78-84) imports data from a file located in the MySQL server's upload directory. The output window at the bottom shows a successful execution of the second command, indicating that 3990 rows were affected.

```
72 ENCLOSED BY '"'
73 LINES TERMINATED BY '\n'
74 IGNORE 1 ROWS;
75
76 • SHOW VARIABLES LIKE 'secure_file_priv';
77
78 • LOAD DATA
79 INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/american_users.csv'
80 INTO TABLE users
81 FIELDS TERMINATED BY ','
82 ENCLOSED BY '"'
83 LINES TERMINATED BY '\n'
84 IGNORE 1 ROWS;
85
86 • LOAD DATA
87 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/european_users.csv"
88 INTO TABLE users
89 FIELDS TERMINATED BY ','
90 ENCLOSED BY '"'
91 LINES TERMINATED BY '\n'
92 IGNORE 1 ROWS;
```

Output

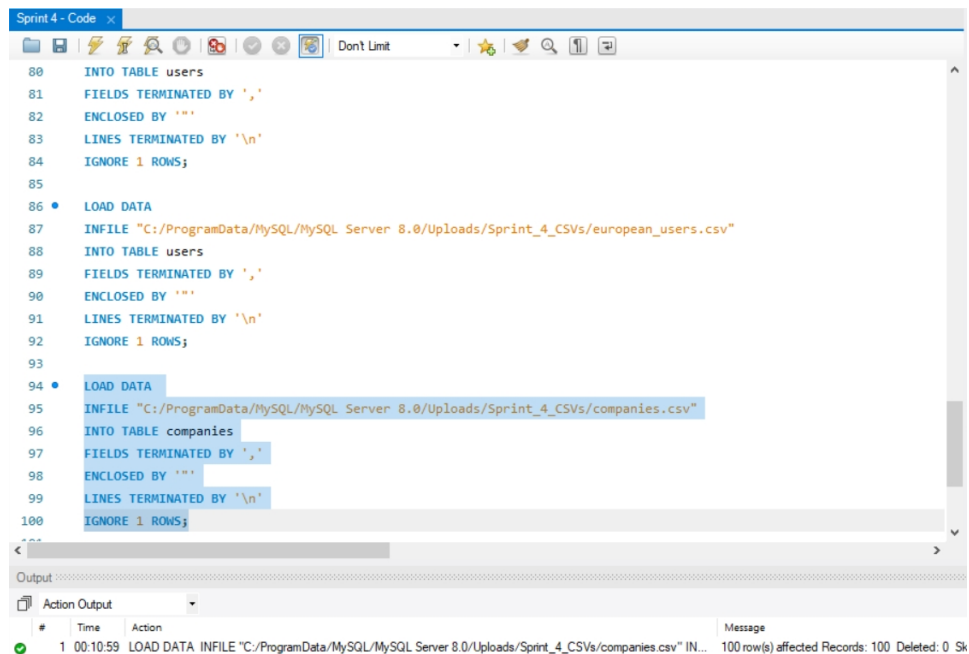
#	Time	Action	Message
1	21:51:29	LOAD DATA INFILE 'C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/european_users.cs...	3990 row(s) affected Records: 3990 Deleted: 0

Fig 10. Importing the CSV data of european_users into the users table

Task S4.01. Database Creation

Mani Rezaeirad

p2p review partner: Damian Vallejos

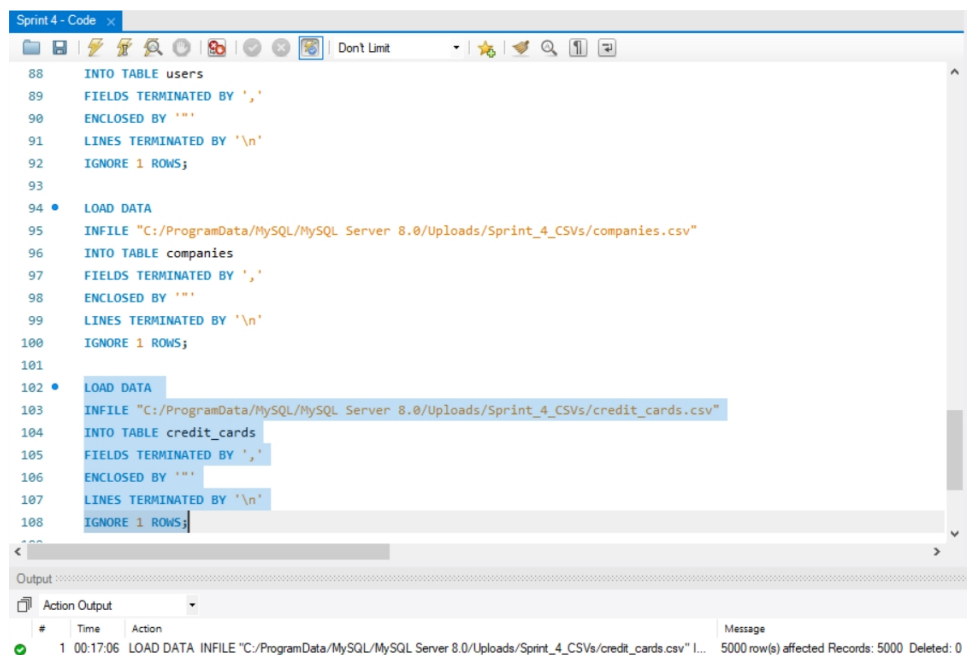


The screenshot shows the SQL Server Enterprise Manager interface. The 'Code' pane displays a SQL script for importing data from a CSV file into the 'companies' table. The script includes a 'LOAD DATA' command with the following parameters: 'INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/companies.csv"', 'INTO TABLE companies', 'FIELDS TERMINATED BY ','', 'ENCLOSED BY ''', 'LINES TERMINATED BY '\n'', and 'IGNORE 1 ROWS;'. The 'Output' pane shows the execution results, indicating that 100 row(s) were affected, with 0 records deleted.

```
80 INTO TABLE users
81 FIELDS TERMINATED BY ','
82 ENCLOSED BY ''
83 LINES TERMINATED BY '\n'
84 IGNORE 1 ROWS;
85
86 • LOAD DATA
87 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/european_users.csv"
88 INTO TABLE users
89 FIELDS TERMINATED BY ','
90 ENCLOSED BY ''
91 LINES TERMINATED BY '\n'
92 IGNORE 1 ROWS;
93
94 • LOAD DATA
95 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/companies.csv"
96 INTO TABLE companies
97 FIELDS TERMINATED BY ','
98 ENCLOSED BY ''
99 LINES TERMINATED BY '\n'
100 IGNORE 1 ROWS;
```

#	Time	Action	Message
1	00:10:59	LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/companies.csv" IN...	100 row(s) affected Records: 100 Deleted: 0 Sk

Fig 11. Importing the CSV data of companies into the companies table



The screenshot shows the SQL Server Enterprise Manager interface. The 'Code' pane displays a SQL script for importing data from a CSV file into the 'credit_cards' table. The script includes a 'LOAD DATA' command with the following parameters: 'INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/credit_cards.csv"', 'INTO TABLE credit_cards', 'FIELDS TERMINATED BY ','', 'ENCLOSED BY ''', 'LINES TERMINATED BY '\n'', and 'IGNORE 1 ROWS;'. The 'Output' pane shows the execution results, indicating that 5000 row(s) were affected, with 0 records deleted.

```
88 INTO TABLE users
89 FIELDS TERMINATED BY ','
90 ENCLOSED BY ''
91 LINES TERMINATED BY '\n'
92 IGNORE 1 ROWS;
93
94 • LOAD DATA
95 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/companies.csv"
96 INTO TABLE companies
97 FIELDS TERMINATED BY ','
98 ENCLOSED BY ''
99 LINES TERMINATED BY '\n'
100 IGNORE 1 ROWS;
101
102 • LOAD DATA
103 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/credit_cards.csv"
104 INTO TABLE credit_cards
105 FIELDS TERMINATED BY ','
106 ENCLOSED BY ''
107 LINES TERMINATED BY '\n'
108 IGNORE 1 ROWS;
```

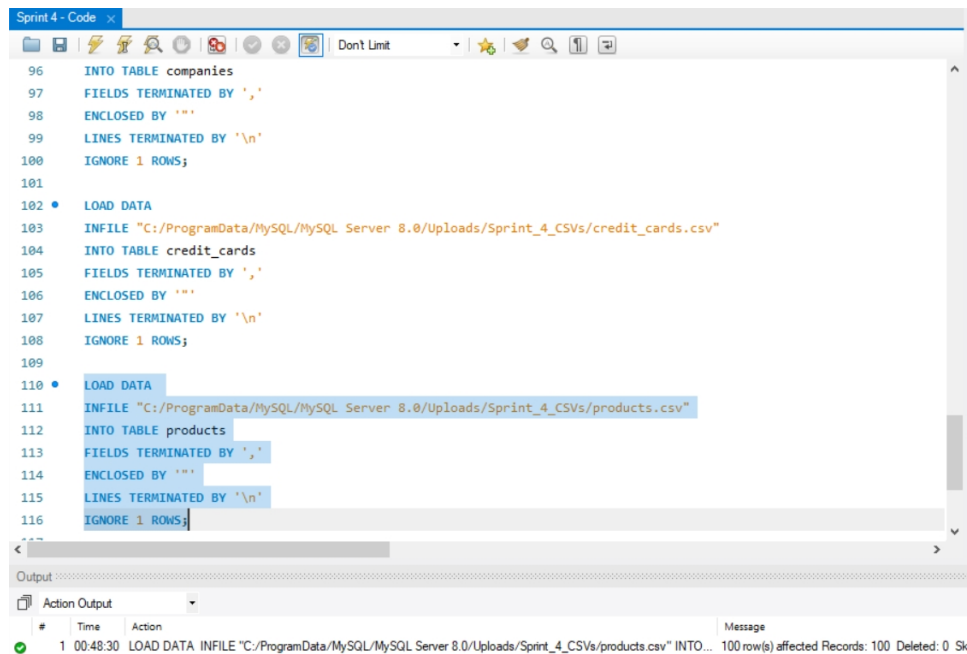
#	Time	Action	Message
1	00:17:06	LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/credit_cards.csv" I...	5000 row(s) affected Records: 5000 Deleted: 0

Fig 12. Importing the CSV data of credit_cards into the credit_cards table

Task S4.01. Database Creation

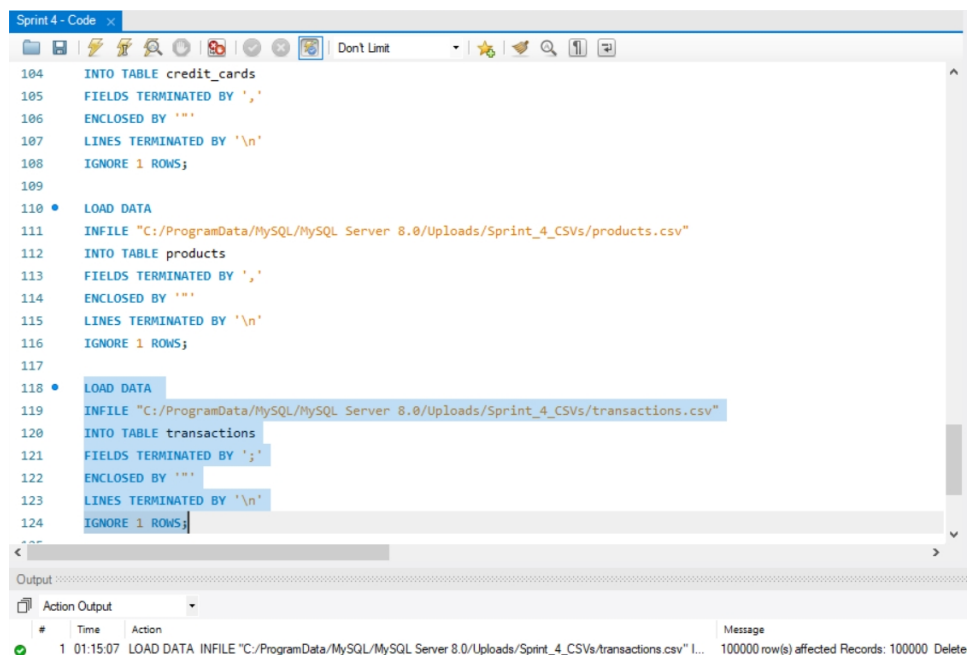
Mani Rezaeirad

p2p review partner: Damian Vallejos



```
96 INTO TABLE companies
97 FIELDS TERMINATED BY ','
98 ENCLOSED BY '"'
99 LINES TERMINATED BY '\n'
100 IGNORE 1 ROWS;
101
102 • LOAD DATA
103 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/credit_cards.csv"
104 INTO TABLE credit_cards
105 FIELDS TERMINATED BY ','
106 ENCLOSED BY '"'
107 LINES TERMINATED BY '\n'
108 IGNORE 1 ROWS;
109
110 • LOAD DATA
111 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/products.csv"
112 INTO TABLE products
113 FIELDS TERMINATED BY ','
114 ENCLOSED BY '"'
115 LINES TERMINATED BY '\n'
116 IGNORE 1 ROWS;
117
Output
Action Output
# Time Action Message
1 00:48:30 LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/products.csv" INTO... 100 row(s) affected Records: 100 Deleted: 0 Sk
```

Fig 13. Importing the CSV data of products into the *products* table



```
104 INTO TABLE credit_cards
105 FIELDS TERMINATED BY ','
106 ENCLOSED BY '"'
107 LINES TERMINATED BY '\n'
108 IGNORE 1 ROWS;
109
110 • LOAD DATA
111 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/products.csv"
112 INTO TABLE products
113 FIELDS TERMINATED BY ','
114 ENCLOSED BY '"'
115 LINES TERMINATED BY '\n'
116 IGNORE 1 ROWS;
117
118 • LOAD DATA
119 INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/transactions.csv"
120 INTO TABLE transactions
121 FIELDS TERMINATED BY ';'
122 ENCLOSED BY '"'
123 LINES TERMINATED BY '\n'
124 IGNORE 1 ROWS;
125
Output
Action Output
# Time Action Message
1 01:15:07 LOAD DATA INFILE "C:/ProgramData/MySQL/MySQL Server 8.0/Uploads/Sprint_4_CSVs/transactions.csv" I... 100000 row(s) affected Records: 100000 Delete
```

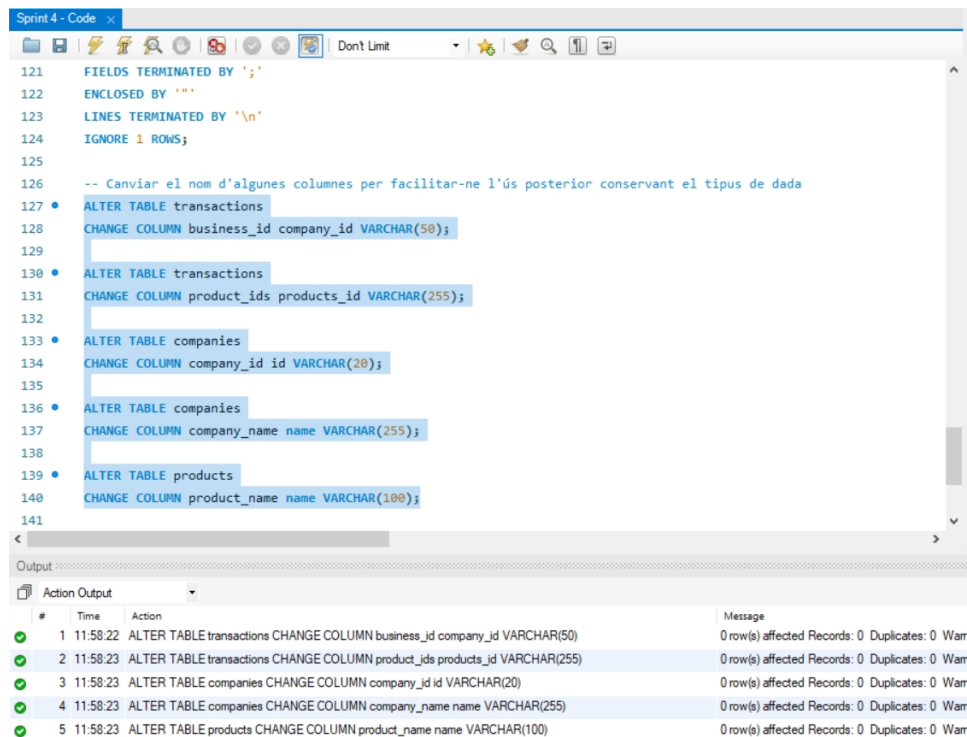
Fig 14. Importing the CSV data of transactions into the *transactions* table

To normalize the headers of each table for further use, some names have been changed, as shown in Figure 15.

Task S4.01. Database Creation

Mani Rezaeirad

p2p review partner: Damian Vallejos



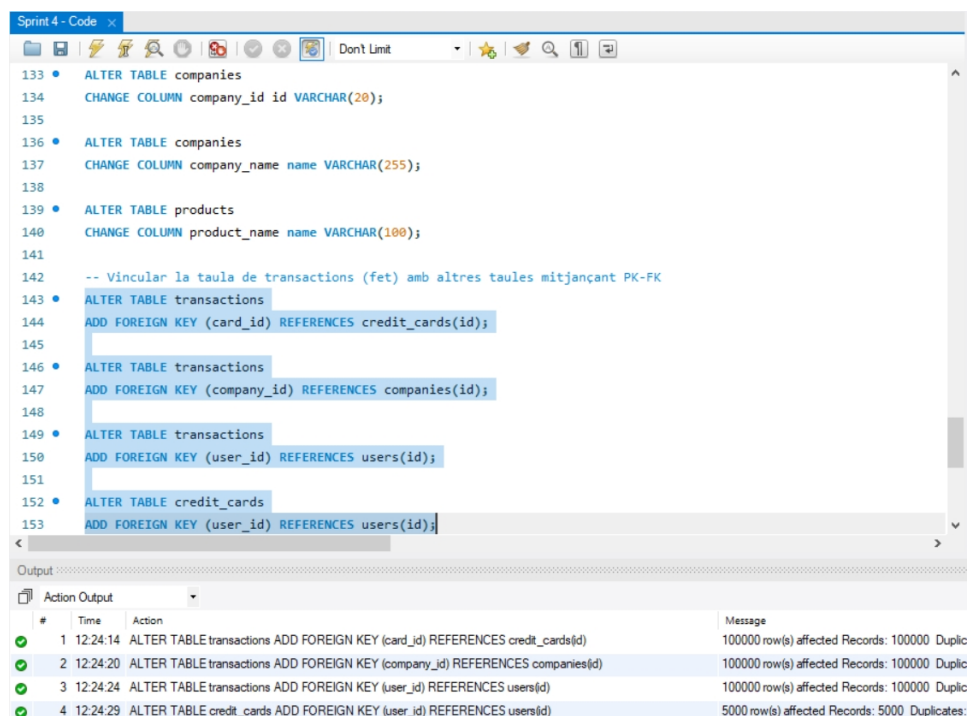
The screenshot shows a SQL editor window titled 'Sprint 4 - Code' with a toolbar at the top. The code area contains SQL statements for altering tables. The output window at the bottom shows the execution results of these statements.

```
121 FIELDS TERMINATED BY ';'
122 ENCLOSED BY ''''
123 LINES TERMINATED BY '\n'
124 IGNORE 1 ROWS;
125
126 -- Canviar el nom d'algunes columnes per facilitar-ne l'ús posterior conservant el tipus de dada
127 ALTER TABLE transactions
128 CHANGE COLUMN business_id company_id VARCHAR(50);
129
130 ALTER TABLE transactions
131 CHANGE COLUMN product_ids products_id VARCHAR(255);
132
133 ALTER TABLE companies
134 CHANGE COLUMN company_id id VARCHAR(20);
135
136 ALTER TABLE companies
137 CHANGE COLUMN company_name name VARCHAR(255);
138
139 ALTER TABLE products
140 CHANGE COLUMN product_name name VARCHAR(100);
141
```

#	Time	Action	Message
1	11:58:22	ALTER TABLE transactions CHANGE COLUMN business_id company_id VARCHAR(50)	0 row(s) affected Records: 0 Duplicates: 0 Warr
2	11:58:23	ALTER TABLE transactions CHANGE COLUMN product_ids products_id VARCHAR(255)	0 row(s) affected Records: 0 Duplicates: 0 Warr
3	11:58:23	ALTER TABLE companies CHANGE COLUMN company_id id VARCHAR(20)	0 row(s) affected Records: 0 Duplicates: 0 Warr
4	11:58:23	ALTER TABLE companies CHANGE COLUMN company_name name VARCHAR(255)	0 row(s) affected Records: 0 Duplicates: 0 Warr
5	11:58:23	ALTER TABLE products CHANGE COLUMN product_name name VARCHAR(100)	0 row(s) affected Records: 0 Duplicates: 0 Warr

Fig 15. Further use reference facilitation by changing some header names

Now we link the transaction table to its corresponding tables, and also, the credit_cards table to the users table using PK-FK, as shown in Figure 16.



The screenshot shows a SQL editor window titled 'Sprint 4 - Code' with a toolbar at the top. The code area contains SQL statements for adding foreign keys. The output window at the bottom shows the execution results of these statements.

```
133 ALTER TABLE companies
134 CHANGE COLUMN company_id id VARCHAR(20);
135
136 ALTER TABLE companies
137 CHANGE COLUMN company_name name VARCHAR(255);
138
139 ALTER TABLE products
140 CHANGE COLUMN product_name name VARCHAR(100);
141
142 -- Vincular la taula de transactions (fet) amb altres taules mitjançant PK-FK
143 ALTER TABLE transactions
144 ADD FOREIGN KEY (card_id) REFERENCES credit_cards(id);
145
146 ALTER TABLE transactions
147 ADD FOREIGN KEY (company_id) REFERENCES companies(id);
148
149 ALTER TABLE transactions
150 ADD FOREIGN KEY (user_id) REFERENCES users(id);
151
152 ALTER TABLE credit_cards
153 ADD FOREIGN KEY (user_id) REFERENCES users(id);
```

#	Time	Action	Message
1	12:24:14	ALTER TABLE transactions ADD FOREIGN KEY (card_id) REFERENCES credit_cards(id)	100000 row(s) affected Records: 100000 Duplic
2	12:24:20	ALTER TABLE transactions ADD FOREIGN KEY (company_id) REFERENCES companies(id)	100000 row(s) affected Records: 100000 Duplic
3	12:24:24	ALTER TABLE transactions ADD FOREIGN KEY (user_id) REFERENCES users(id)	100000 row(s) affected Records: 100000 Duplic
4	12:24:29	ALTER TABLE credit_cards ADD FOREIGN KEY (user_id) REFERENCES users(id)	5000 row(s) affected Records: 5000 Duplicates:

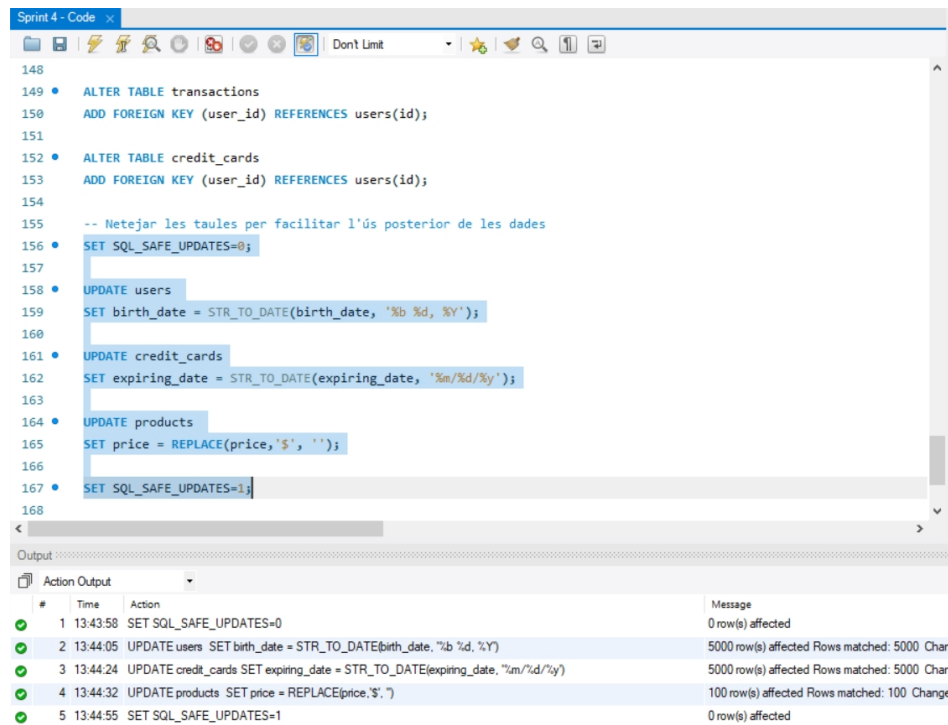
Fig 16. Linking the transactions table to other tables using PK-FK

Task S4.01. Database Creation

Mani Rezaeirad

p2p review partner: Damian Vallejos

Before entering the first exercise, some last small database cleaning steps are done, as shown in Figure 17. To avoid the safe updates error, we turn updates off by setting it to 0, then set it back to 1 after running the code (NB! it is not recommended by many users, but it was the fastest way to solve the problem).



```
148
149 • ALTER TABLE transactions
150 ADD FOREIGN KEY (user_id) REFERENCES users(id);
151
152 • ALTER TABLE credit_cards
153 ADD FOREIGN KEY (user_id) REFERENCES users(id);
154
155 -- Netejar les taules per facilitar l'ús posterior de les dades
156 SET SQL_SAFE_UPDATES=0;
157
158 • UPDATE users
159 SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y');
160
161 • UPDATE credit_cards
162 SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y');
163
164 • UPDATE products
165 SET price = REPLACE(price, '$', '');
166
167 • SET SQL_SAFE_UPDATES=1;
168
```

#	Time	Action	Message
1	13:43:58	SET SQL_SAFE_UPDATES=0	0 row(s) affected
2	13:44:05	UPDATE users SET birth_date = STR_TO_DATE(birth_date, '%b %d, %Y')	5000 row(s) affected Rows matched: 5000 Chan
3	13:44:24	UPDATE credit_cards SET expiring_date = STR_TO_DATE(expiring_date, '%m/%d/%y')	5000 row(s) affected Rows matched: 5000 Chan
4	13:44:32	UPDATE products SET price = REPLACE(price, '\$', '')	100 row(s) affected Rows matched: 100 Change
5	13:44:55	SET SQL_SAFE_UPDATES=1	0 row(s) affected

Fig 17. Cleaning the database by changing some formats of some table columns

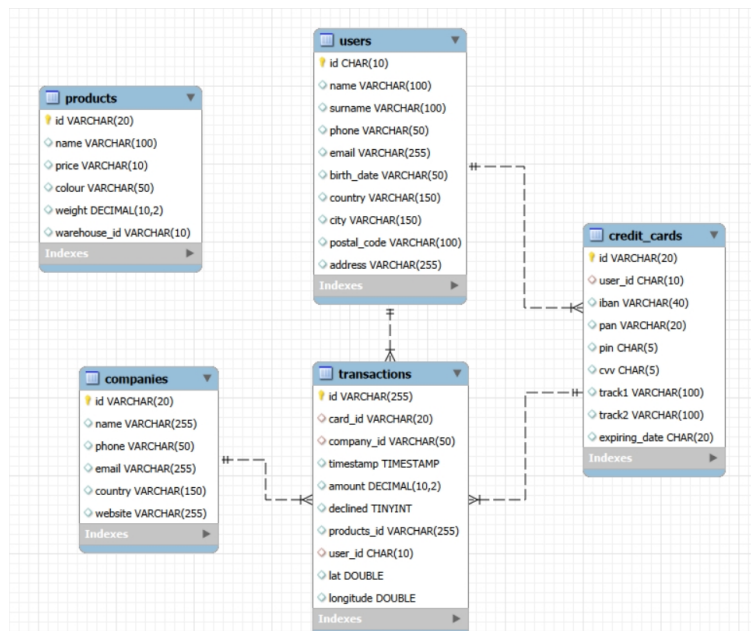


Fig 18. ER diagram of the sprint4_bbdd database with a STAR schema

Task S4.01. Database Creation

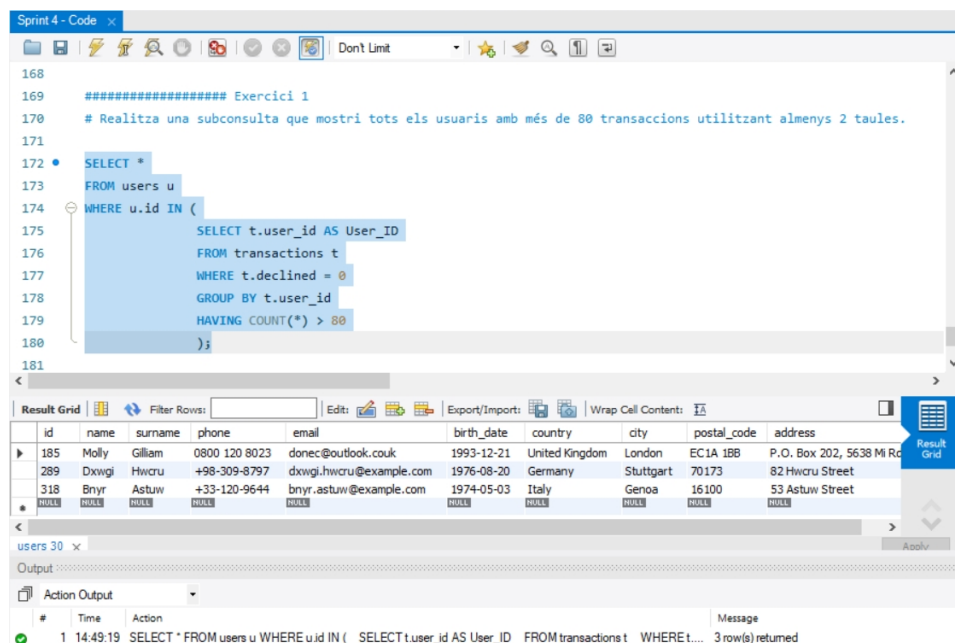
Mani Rezaeirad

p2p review partner: Damian Vallejos

As can be seen in Figure 18, a STAR schema is depicted, with the transactions table (fact table) in the middle and the users, credit_cards, and companies tables (dimension tables) in a 1:N relationship with it. The users table also has a 1:N relationship with the credit_cards table.

Exercise 1

Perform a subquery that shows all users with more than 80 transactions using at least 2 tables.



The screenshot shows a database IDE with a SQL query editor and a results grid. The query is as follows:

```
168
169 ##### Exercici 1
170 # Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.
171
172 SELECT *
173 FROM users u
174 WHERE u.id IN (
175     SELECT t.user_id AS User_ID
176     FROM transactions t
177     WHERE t.declined = 0
178     GROUP BY t.user_id
179     HAVING COUNT(*) > 80
180 )
181
```

The results grid shows the following data:

id	name	surname	phone	email	birth_date	country	city	postal_code	address
185	Molly	Gilliam	0800 120 8023	donec@outlook.co.uk	1993-12-21	United Kingdom	London	EC1A 1BB	P.O. Box 202, 5638 MI Rd
289	Dxwgi	Hwrcu	+98-309-8797	dxwgi.hwrcu@example.com	1976-08-20	Germany	Stuttgart	70173	82 Hwrcu Street
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	1974-05-03	Italy	Genoa	16100	53 Astuw Street

The Action Output section shows the following message:

```
1 14:49:19 SELECT * FROM users u WHERE u.id IN ( SELECT t.user_id AS User_ID FROM transactions t WHERE t.declined = 0 GROUP BY t.user_id HAVING COUNT(*) > 80 ) 3 row(s) returned
```

Fig 19. Users with more than 80 transactions

To solve this exercise, two tables, users and transactions, are used. Remember, if the command line “t.declined = 0” is omitted, there will be 4 users instead!

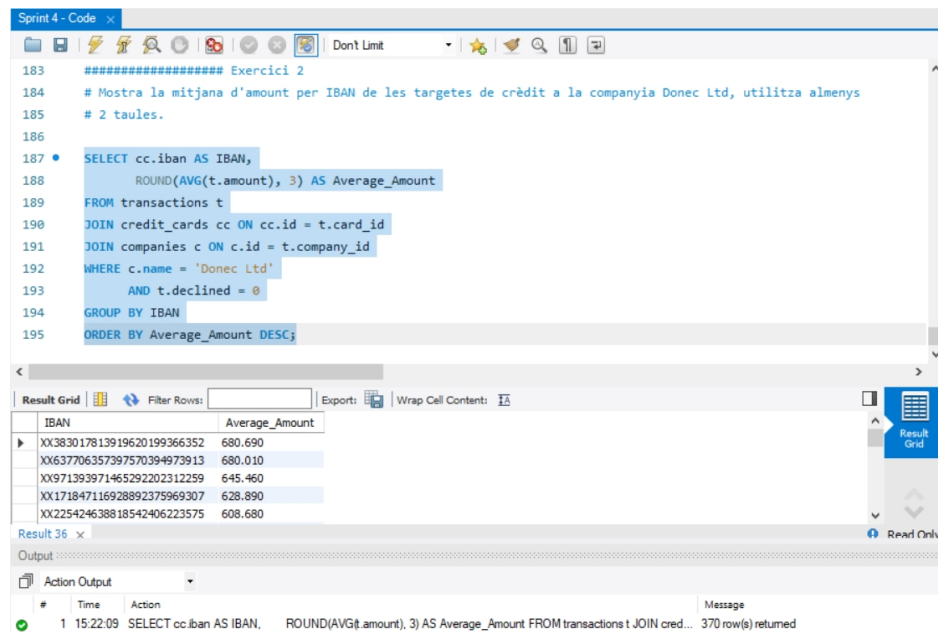
Exercise 2

Show the average amount per IBAN of credit cards in the company Donec Ltd. Use at least 2 tables.

Task S4.01. Database Creation

Mani Rezaeirad

p2p review partner: Damian Vallejos



```
183 ##### Exercici 2
184 # Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys
185 # 2 taules.
186
187 • SELECT cc.iban AS IBAN,
188       ROUND(AVG(t.amount), 3) AS Average_Amount
189 FROM transactions t
190 JOIN credit_cards cc ON cc.id = t.card_id
191 JOIN companies c ON c.id = t.company_id
192 WHERE c.name = 'Donec Ltd'
193       AND t.declined = 0
194 GROUP BY IBAN
195 ORDER BY Average_Amount DESC;
```

IBAN	Average_Amount
XX383017813919620199366352	680.690
XX637706357397570394973913	680.010
XX971393971465292202312259	645.460
XX171847116928892375969307	628.890
XX225424638818542406223575	608.680

Result 36 x

Output

Action Output

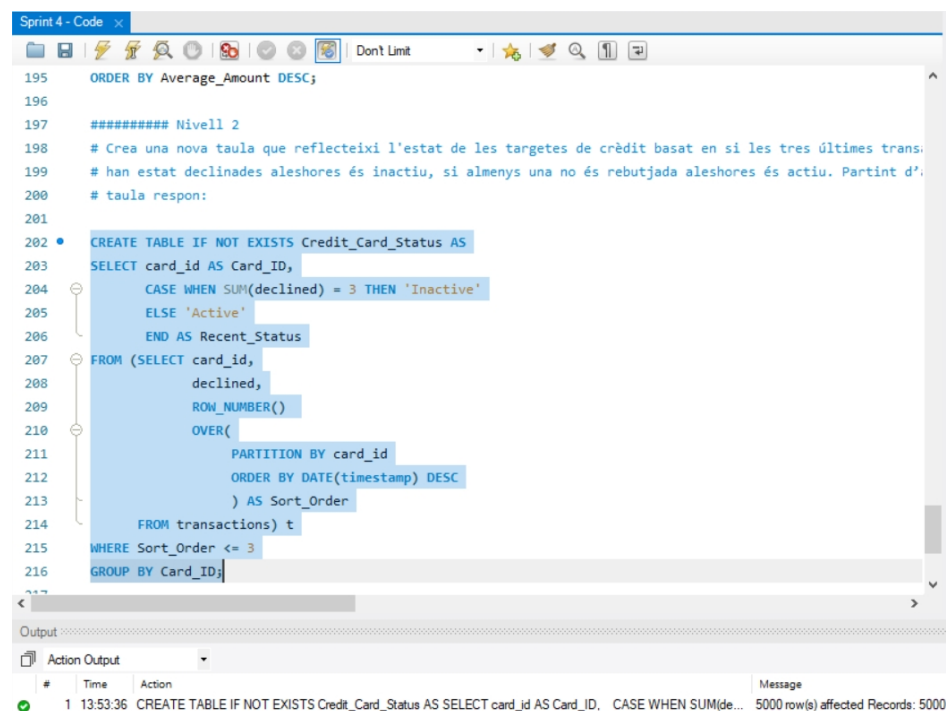
Time Action Message

1 15:22:09 SELECT cc.iban AS IBAN, ROUND(AVG(t.amount), 3) AS Average_Amount FROM transactions t JOIN cred... 370 row(s) returned

Fig 20. Average amount per IBAN of credit cards in the company Donec Ltd

Level 2

Create a new table that reflects the status of credit cards based on whether the last three transactions have been declined then it is inactive, if at least one is not declined then it is active. Based on this table answer:



```
195 ORDER BY Average_Amount DESC;
196
197 ##### Nivell 2
198 # Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes trans:
199 # han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d':
200 # taula respon:
201
202 • CREATE TABLE IF NOT EXISTS Credit_Card_Status AS
203   SELECT card_id AS Card_ID,
204         CASE WHEN SUM(declined) = 3 THEN 'Inactive'
205              ELSE 'Active'
206         END AS Recent_Status
207 FROM (SELECT card_id,
208             declined,
209             ROW_NUMBER()
210             OVER(
211               PARTITION BY card_id
212               ORDER BY DATE(timestamp) DESC
213             ) AS Sort_Order
214        FROM transactions) t
215 WHERE Sort_Order <= 3
216 GROUP BY Card_ID;
```

Output

Action Output

Time Action Message

1 13:53:36 CREATE TABLE IF NOT EXISTS Credit_Card_Status AS SELECT card_id AS Card_ID, CASE WHEN SUM(de... 5000 row(s) affected Records: 5000

Fig 21. Creating a new table for credit card status

Task S4.01. Database Creation

Mani Rezaeirad

p2p review partner: Damian Vallejos

Exercise 1

How many cards are active?

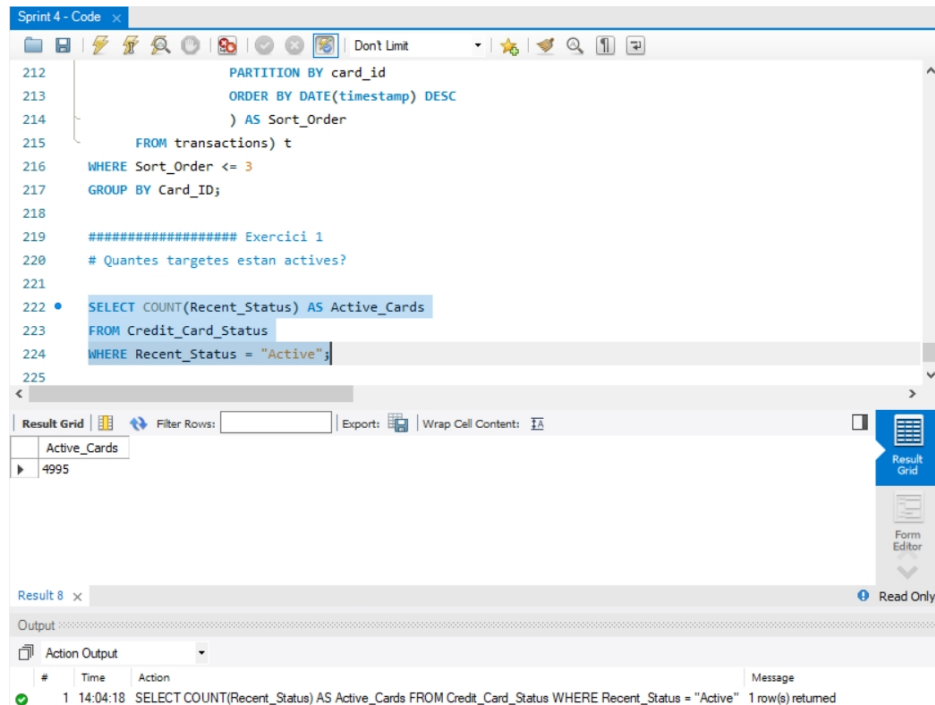


Fig 22. Number of active credit cards

Level 3

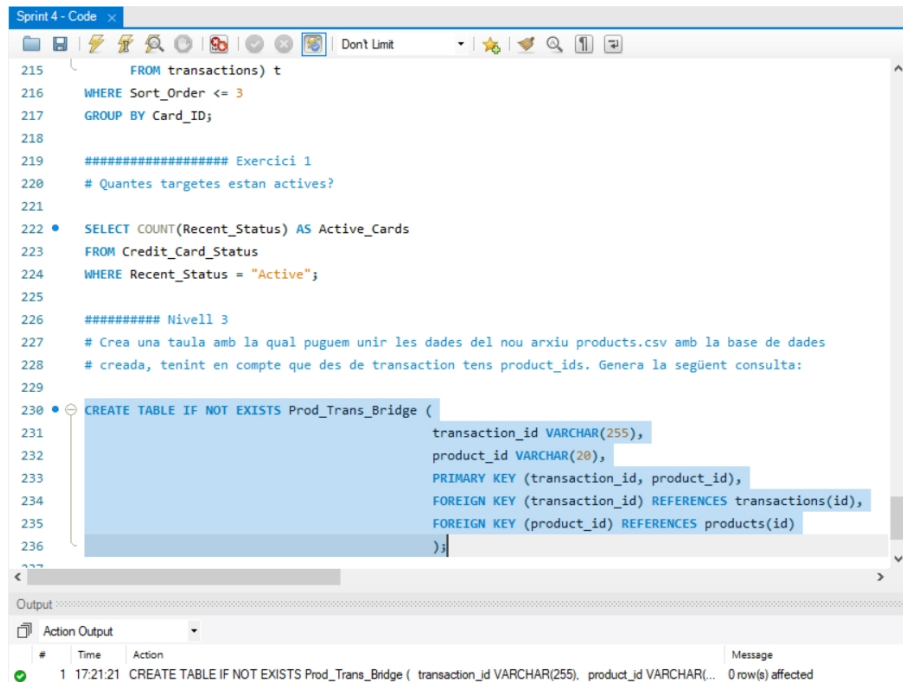
Create a table with which we can join the data from the new products.csv file with the created database, taking into account that from transactions you have product_ids. Generate the following query:

To do this, we need a bridge table that relates the products table to the transactions table. This phase is depicted in Figure 23. Then, we need to change the format of the transactions.product_id to JSON. It is done as illustrated in Figure 24.

Task S4.01. Database Creation

Mani Rezaeirad

p2p review partner: Damian Vallejos

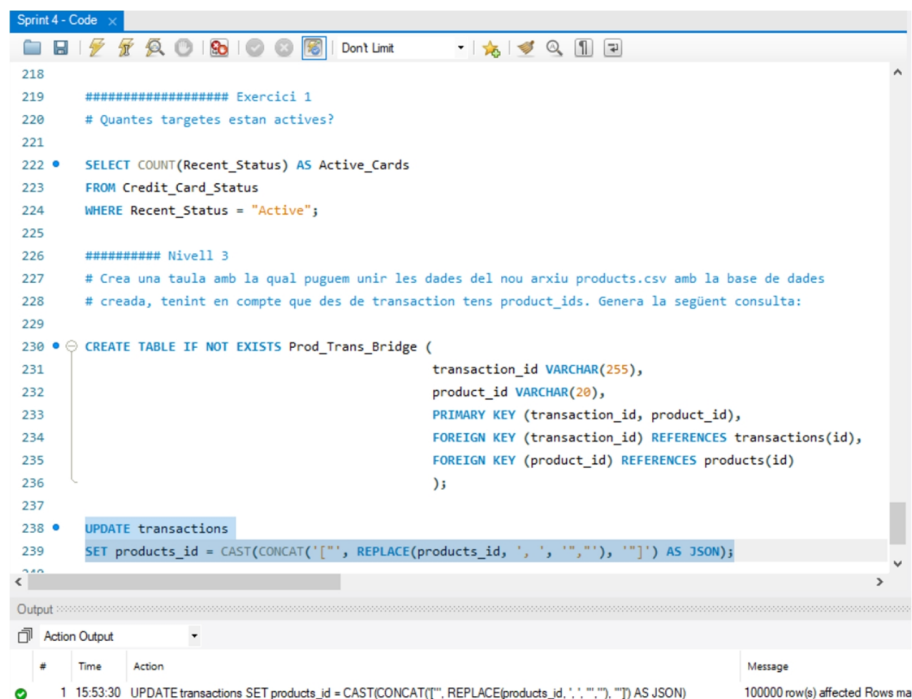


```
Sprint4 - Code
215 FROM transactions) t
216 WHERE Sort_Order <= 3
217 GROUP BY Card_ID;
218
219 ##### Exercici 1
220 # Quantes targetes estan actives?
221
222 • SELECT COUNT(Recent_Status) AS Active_Cards
223 FROM Credit_Card_Status
224 WHERE Recent_Status = "Active";
225
226 ##### Nivell 3
227 # Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades
228 # creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:
229
230 • CREATE TABLE IF NOT EXISTS Prod_Trans_Bridge (
231     transaction_id VARCHAR(255),
232     product_id VARCHAR(20),
233     PRIMARY KEY (transaction_id, product_id),
234     FOREIGN KEY (transaction_id) REFERENCES transactions(id),
235     FOREIGN KEY (product_id) REFERENCES products(id)
236 );
```

Output

#	Time	Action	Message
1	17:21:21	CREATE TABLE IF NOT EXISTS Prod_Trans_Bridge (transaction_id VARCHAR(255), product_id VARCHAR(20), PRIMARY KEY (transaction_id, product_id), FOREIGN KEY (transaction_id) REFERENCES transactions(id), FOREIGN KEY (product_id) REFERENCES products(id));	0 row(s) affected

Fig 23. Creating the bridge table between the users table and the transactions table



```
Sprint4 - Code
218
219 ##### Exercici 1
220 # Quantes targetes estan actives?
221
222 • SELECT COUNT(Recent_Status) AS Active_Cards
223 FROM Credit_Card_Status
224 WHERE Recent_Status = "Active";
225
226 ##### Nivell 3
227 # Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades
228 # creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:
229
230 • CREATE TABLE IF NOT EXISTS Prod_Trans_Bridge (
231     transaction_id VARCHAR(255),
232     product_id VARCHAR(20),
233     PRIMARY KEY (transaction_id, product_id),
234     FOREIGN KEY (transaction_id) REFERENCES transactions(id),
235     FOREIGN KEY (product_id) REFERENCES products(id)
236 );
237
238 • UPDATE transactions
239 SET products_id = CAST(CONCAT('["', REPLACE(products_id, ',', '","'), "']') AS JSON);
```

Output

#	Time	Action	Message
1	15:53:30	UPDATE transactions SET products_id = CAST(CONCAT('["', REPLACE(products_id, ',', '","'), "']') AS JSON);	100000 row(s) affected Rows ma

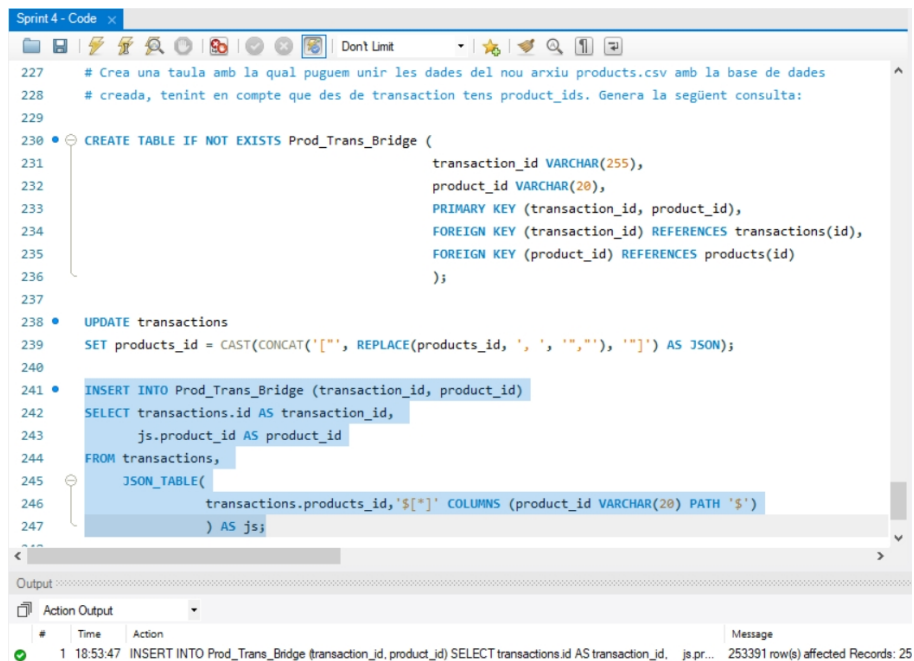
Fig 24. Changing the format of a specific column to JSON

Now, it is time to insert the corresponding data into our new table, using a function called `JSON_TABLE`, as demonstrated in Figure 25.

Task S4.01. Database Creation

Mani Rezaeirad

p2p review partner: Damian Vallejos



```
227 # Crea una taula amb la qual puguem unir les dades del nou arxiu products.csv amb la base de dades
228 # creada, tenint en compte que des de transaction tens product_ids. Genera la següent consulta:
229
230 CREATE TABLE IF NOT EXISTS Prod_Trans_Bridge (
231     transaction_id VARCHAR(255),
232     product_id VARCHAR(20),
233     PRIMARY KEY (transaction_id, product_id),
234     FOREIGN KEY (transaction_id) REFERENCES transactions(id),
235     FOREIGN KEY (product_id) REFERENCES products(id)
236 );
237
238 UPDATE transactions
239 SET products_id = CAST(CONCAT('["', REPLACE(products_id, ', ', '","'), "']') AS JSON);
240
241 INSERT INTO Prod_Trans_Bridge (transaction_id, product_id)
242 SELECT transactions.id AS transaction_id,
243        js.product_id AS product_id
244 FROM transactions,
245     JSON_TABLE(
246         transactions.products_id, '$[*]' COLUMNS (product_id VARCHAR(20) PATH '$')
247     ) AS js;
```

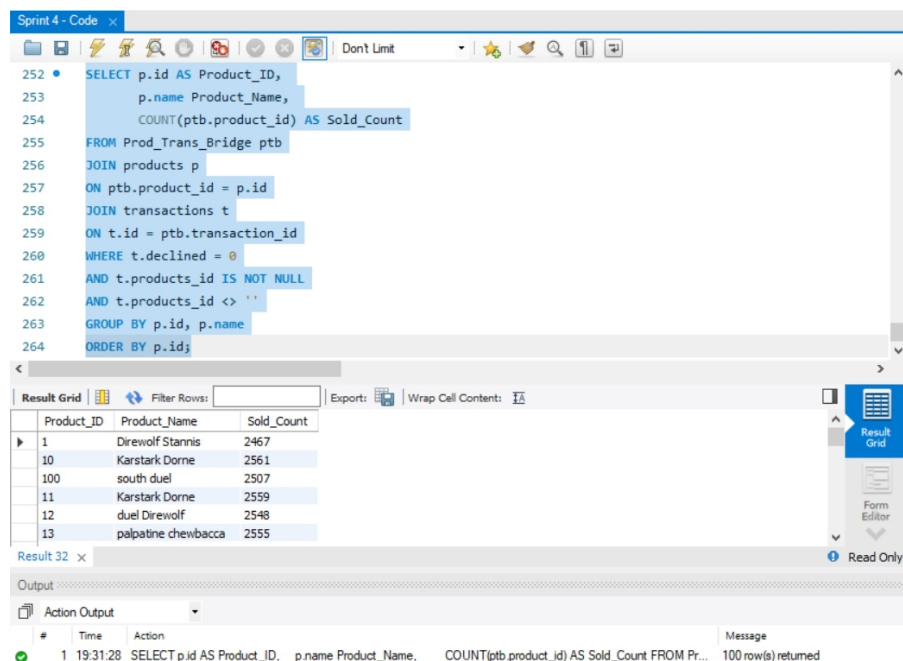
Output

#	Time	Action	Message
1	18:53:47	INSERT INTO Prod_Trans_Bridge (transaction_id, product_id) SELECT transactions.id AS transaction_id, js.pr...	253391 row(s) affected Records: 25

Fig 25. Inserting data into the new table using the JSON_TABLE function

Exercise 1

We need to know the number of times each product has been sold.



```
252 SELECT p.id AS Product_ID,
253        p.name Product_Name,
254        COUNT(ptb.product_id) AS Sold_Count
255 FROM Prod_Trans_Bridge ptb
256 JOIN products p
257 ON ptb.product_id = p.id
258 JOIN transactions t
259 ON t.id = ptb.transaction_id
260 WHERE t.declined = 0
261 AND t.products_id IS NOT NULL
262 AND t.products_id <> ''
263 GROUP BY p.id, p.name
264 ORDER BY p.id;
```

Result Grid

	Product_ID	Product_Name	Sold_Count
▶	1	Direwolf Stannis	2467
	10	Karstark Dorne	2561
	100	south duel	2507
	11	Karstark Dorne	2559
	12	duel Direwolf	2548
	13	palpatine chewbacca	2555

Output

#	Time	Action	Message
1	19:31:28	SELECT p.id AS Product_ID, p.name Product_Name, COUNT(ptb product_id) AS Sold_Count FROM Pr...	100 row(s) returned

Fig 26. The number of times each product has been sold

Given all the available information, the number of sales per product is simply a matter of using JOIN function, as shown in Figure 26.

Task S4.01. Database Creation

Mani Rezaeirad

p2p review partner: Damian Vallejos

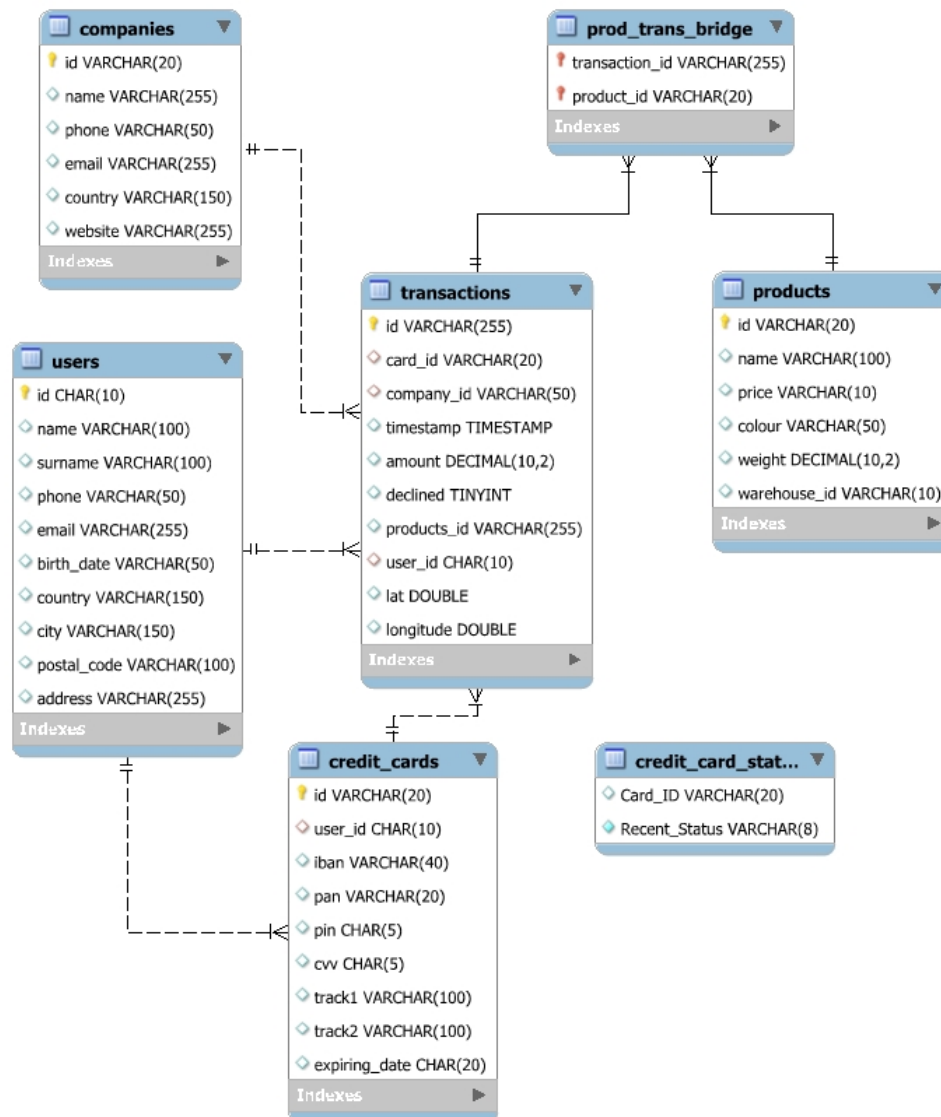


Fig 27. Updated ER diagram of the sprint4_bbdd database

Finally, Figure 27 shows the updated ER diagram of our database, which, unlike Figure 18, has a snowflake schema. As mentioned earlier, the `Prod_Trans_Bridge` table creates an M:N relationship between the `transactions` table and the `products` table. Note that since no exercise has asked to link the `credit_card_status` table to the `credit_cards` table, it is left without any connection.